

STEM BUILDERS



More Functions

8. More Functions

Earlier we learned the most bare-boned versions of functions. In this section we will learn more general concepts about functions, such as how to use functions to return values, and how to pass different kinds of data structures between functions.

Default argument values

When we first introduced functions, we started with this example:

```
def thank_you(name):  
    # This function prints a two-line personalized thank you message.  
    print("\nYou are doing good work, %s!" % name)  
    print("Thank you very much for your efforts on this project.")
```

```
thank_you('Adriana')  
thank_you('Billy')  
thank_you('Caroline')
```

Output:

```
You are doing good work, Adriana!  
Thank you very much for your efforts on this project.
```

```
You are doing good work, Billy!  
Thank you very much for your efforts on this project.
```

```
You are doing good work, Caroline!  
Thank you very much for your efforts on this project.
```

This function works fine, but it fails if you don't pass in a value:

```
def thank_you(name):  
    # This function prints a two-line personalized thank you message.  
    print("\nYou are doing good work, %s!" % name)  
    print("Thank you very much for your efforts on this project.")
```

```
thank_you('Adriana')  
thank_you('Billy')  
thank_you('Caroline')  
thank_you()
```

Output:

----- **TypeError**

```
Traceback (most recent call last)
<ipython-input-2-a964f1e96d9b> in <module>()
      7 thank_you('Billy')
      8 thank_you('Caroline')
----> 9 thank_you()
```

TypeError: thank_you() takes exactly 1 argument (0 given)

You are doing good work, Adriana!
Thank you very much for your efforts on this project.

You are doing good work, Billy!
Thank you very much for your efforts on this project.

You are doing good work, Caroline!
Thank you very much for your efforts on this project.

That makes sense; the function needs to have a name in order to do its work, so without a name it is stuck.

If you want your function to do something by default, even if no information is passed to it, you can do so by giving your arguments default values. You do this by specifying the default values when you define the function:

```
def thank_you(name='everyone'):
    # This function prints a two-line personalized thank you message.
    # If no name is passed in, it prints a general thank you message
    # to everyone.
    print("\nYou are doing good work, %s!" % name)
    print("Thank you very much for your efforts on this project.")
```

```
thank_you('Adriana')
thank_you('Billy')
thank_you('Caroline')
thank_you()
```

Output:

You are doing good work, Adriana!
Thank you very much for your efforts on this project.

You are doing good work, Billy!
Thank you very much for your efforts on this project.

You are doing good work, Caroline!
Thank you very much for your efforts on this project.

You are doing good work, everyone!
Thank you very much for your efforts on this project.

This is particularly useful when you have a number of arguments in your function, and some of those arguments almost always have the same value. This allows people who use the function to only specify the values that are unique to their use of the function.

Exercises

Games

- Write a function that accepts the name of a game and prints a statement such as, "I like playing chess!"
- Give the argument a default value, such as `chess`.
- Call your function at least three times. Make sure at least one of the calls includes an argument, and at least one call includes no arguments.

Favorite Movie

- Write a function that accepts the name of a movie, and prints a statement such as, "My favorite movie is The Princess Bride."
- Give the argument a default value, such as `The Princess Bride`.
- Call your function at least three times. Make sure at least one of the calls includes an argument, and at least one call includes no arguments.

Positional Arguments

Much of what you will have to learn about using functions involves how to pass values from your calling statement to the function itself. The example we just looked at is pretty simple, in that the function only needed one argument in order to do its work. Let's take a look at a function that requires two arguments to do its work.

Let's make a simple function that takes in three arguments. Let's make a function that takes in a person's first and last name, and then prints out everything it knows about the person.

Here is a simple implementation of this function:

```
def describe_person(first_name, last_name, age):  
    # This function takes in a person's first and last name,  
    # and their age.  
    # It then prints this information out in a simple format.  
    print("First name: %s" % first_name.title())  
    print("Last name: %s" % last_name.title())  
    print("Age: %d\n" % age)
```

```
describe_person('brian', 'kernighan', 71)
describe_person('ken', 'thompson', 70)
describe_person('adele', 'goldberg', 68)
```

Output:

```
First name: Brian
Last name: Kernighan
Age: 71
```

```
First name: Ken
Last name: Thompson
Age: 70
```

```
First name: Adele
Last name: Goldberg
Age: 68
```

The arguments in this function are `first_name`, `last_name`, and `age`. These are called *positional arguments* because Python knows which value to assign to each by the order in which you give the function values. In the calling line

```
describe_person('brian', 'kernighan', 71)
```

we send the values *brian*, *kernighan*, and *71* to the function. Python matches the first value *brian* with the first argument `first_name`. It matches the second value *kernighan* with the second argument `last_name`. Finally it matches the third value *71* with the third argument `age`.

This is pretty straightforward, but it means we have to make sure to get the arguments in the right order. If we mess up the order, we get an error:

```
def describe_person(first_name, last_name, age):
    # This function takes in a person's first and last name,
    # and their age.
    # It then prints this information out in a simple format.
    print("First name: %s" % first_name.title())
    print("Last name: %s" % last_name.title())
    print("Age: %d\n" % age)
```

```
describe_person(71, 'brian', 'kernighan')
describe_person(70, 'ken', 'thompson')
describe_person(68, 'adele', 'goldberg')
```

Output:

```
----- AttributeError
r
Traceback (most recent call last)
<ipython-input-23-59fb6d0341c1> in <module>()
      7     print("Age: %d\n" % age)
```

```
8
----> 9 describe_person(71, 'brian', 'kernighan')
10 describe_person(70, 'ken', 'thompson')
11 describe_person(68, 'adele', 'goldberg')

<ipython-input-23-59fb6d0341c1> in describe_person(first_name, last_name, age)
3     # and their age.
4     # It then prints this information out in a simple format.
----> 5     print("First name: %s" % first_name.title())
6     print("Last name: %s" % last_name.title())
7     print("Age: %d\n" % age)
```

AttributeError: 'int' object has no attribute 'title'

This fails because Python tries to match the value 71 with the argument `first_name`, the value *brian* with the argument `last_name`, and the value *kernighan* with the argument `age`. Then when it tries to print the value `first_name.title()`, it realizes it can't use the `title()` method on an integer.

Exercises

Favorite Colors

- Write a function that takes two arguments, a person's name and their favorite color. The function should print out a statement such as "Hillary's favorite color is blue."
- Call your function three times, with a different person and color each time.

Phones

- Write a function that takes two arguments, a brand of phone and a model name. The function should print out a phrase such as "iPhone 11".
- Call your function three times, with a different combination of brand and model each time.

Keyword arguments

Python allows us to use a syntax called *keyword arguments*. In this case, we can give the arguments in any order when we call the function, as long as we use the name of the arguments in our calling statement. Here is how the previous code can be made to work using keyword arguments:

```
def describe_person(first_name, last_name, age):  
    # This function takes in a person's first and last name,  
    # and their age.      # It then prints this information out in a simple format.  
    print("First name: %s" % first_name.title())  
    print("Last name: %s" % last_name.title())  
    print("Age: %d\n" % age)
```

```
describe_person(age=71, first_name='brian', last_name='kernighan')  
describe_person(age=70, first_name='ken', last_name='thompson')  
describe_person(age=68, first_name='adele', last_name='goldberg')
```

Output:

```
First name: Brian  
Last name: Kernighan  
Age: 71
```

```
First name: Ken  
Last name: Thompson  
Age: 70
```

```
First name: Adele  
Last name: Goldberg  
Age: 68
```

This works, because Python does not have to match values to arguments by position. It matches the value 71 with the argument `age`, because the value 71 is clearly marked to go with that argument. This syntax is a little more typing, but it makes for very readable code.

Mixing positional and keyword arguments

It can make good sense sometimes to mix positional and keyword arguments. In our previous example, we can expect this function to always take in a first name and a last name. Before we start mixing positional and keyword arguments, let's add another piece of information to our description of a person. Let's also go back to using just positional arguments for a moment:

```
def describe_person(first_name, last_name, age, favorite_language):  
    # This function takes in a person's first and last name,  
    # their age, and their favorite language.  
    # It then prints this information out in a simple format.  
    print("First name: %s" % first_name.title())  
    print("Last name: %s" % last_name.title())  
    print("Age: %d" % age)  
    print("Favorite language: %s\n" % favorite_language)
```

```
describe_person('brian', 'kernighan', 71, 'C')  
describe_person('ken', 'thompson', 70, 'Go')  
describe_person('adele', 'goldberg', 68, 'Smalltalk')
```

Output:

```
First name: Brian  
Last name: Kernighan  
Age: 71  
Favorite language: C
```

```
First name: Ken  
Last name: Thompson  
Age: 70  
Favorite language: Go
```

```
First name: Adele  
Last name: Goldberg  
Age: 68  
Favorite language: Smalltalk
```

We can expect anyone who uses this function to supply a first name and a last name, in that order. But now we are starting to include some information that might not apply to everyone. We can address this by keeping positional arguments for the first name and last name, but expect keyword arguments for everything else. We can show this works by adding a few more people, and having different information about each person:

```
def describe_person(first_name, last_name, age=None, favorite_language=None, died=None)
:
    # This function takes in a person's first and last name,
    # their age, and their favorite language.
    # It then prints this information out in a simple format.
    # Required information:
    print("First name: %s" % first_name.title())
    print("Last name: %s" % last_name.title())

    # Optional information:
    if age:
        print("Age: %d" % age)
    if favorite_language:
        print("Favorite language: %s" % favorite_language)
    if died:
        print("Died: %d" % died)

    # Blank line at end.
    print("\n")

describe_person('brian', 'kernighan', favorite_language='C')
describe_person('ken', 'thompson', age=70)
describe_person('adele', 'goldberg', age=68, favorite_language='Smalltalk')
describe_person('dennis', 'ritchie', favorite_language='C', died=2011)
describe_person('guido', 'van rossum', favorite_language='Python')
```

Output:

First name: Brian
Last name: Kernighan
Favorite language: C

First name: Ken
Last name: Thompson
Age: 70

First name: Adele
Last name: Goldberg
Age: 68
Favorite language: Smalltalk

First name: Dennis
Last name: Ritchie
Favorite language: C
Died: 2011

First name: Guido
Last name: Van Rossum
Favorite language: Python

Everyone needs a first and last name, but everything else is optional. This code takes advantage of the Python keyword `None`, which acts as an empty value for a variable. This way, the user is free to supply any of the 'extra' values they care to. Any arguments that don't receive a value are not displayed. Python matches these extra values by name, rather than by position. This is a very common and useful way to define functions.

Exercises

Sports Teams

- Write a function that takes in two arguments, the name of a city and the name of a sports team from that city.
- Call your function three times, using a mix of positional and keyword arguments.

World Languages

- Write a function that takes in two arguments, the name of a country and a major language spoken there.
- Call your function three times, using a mix of positional and keyword arguments.

Accepting an arbitrary number of arguments

We have now seen that using keyword arguments can allow for much more flexible calling statements.

- This benefits you in your own programs, because you can write one function that can handle many different situations you might encounter.
- This benefits you if other programmers use your programs, because your functions can apply to a wide range of situations.
- This benefits you when you use other programmers' functions, because their functions can apply to many situations you will care about.

There is another issue that we can address, though. Let's consider a function that takes two numbers in, and prints out the sum of the two numbers:

```
def adder(num_1, num_2):  
    # This function adds two numbers together, and prints the sum.  
    sum = num_1 + num_2  
    print("The sum of your numbers is %d." % sum)
```

```
# Let's add some numbers.
```

```
adder(1, 2)
adder(-1, 2)
adder(1, -2)
```

Output:

```
The sum of your numbers is 3.
The sum of your numbers is 1.
The sum of your numbers is -1.
```

This function appears to work well. But what if we pass it three numbers, which is a perfectly reasonable thing to do mathematically?

```
def adder(num_1, num_2):
    # This function adds two numbers together, and prints the sum.
    sum = num_1 + num_2
    print("The sum of your numbers is %d." % sum)
```

```
# Let's add some numbers.
```

```
adder(1, 2, 3)
```

Output:

```
-----
Traceback (most recent call last)
<ipython-input-40-9939998d2a01> in <module>()
      5
      6 # Let's add some numbers.
----> 7 adder(1, 2, 3)
TypeError
```

TypeError: adder() takes exactly 2 arguments (3 given)

This function fails, because no matter what mix of positional and keyword arguments we use, the function is only written to accept two arguments. In fact, a function written in this way will only work with *exactly* two arguments.

Accepting a sequence of arbitrary length

Python gives us a syntax for letting a function accept an arbitrary number of arguments. If we place an argument at the end of the list of arguments, with an asterisk in front of it, that argument will collect any remaining values from the calling statement into a tuple. Here is an example demonstrating how this works:

```
def example_function(arg_1, arg_2, *arg_3):
    # Let's look at the argument values.
    print('\narg_1:', arg_1)
    print('arg_2:', arg_2)
    print('arg_3:', arg_3)
```

```
example_function(1, 2)
example_function(1, 2, 3)
example_function(1, 2, 3, 4)
example_function(1, 2, 3, 4, 5)
```

Output:

```
arg_1: 1
arg_2: 2
arg_3: ()
```

```
arg_1: 1
arg_2: 2
arg_3: (3,)
```

```
arg_1: 1
arg_2: 2
arg_3: (3, 4)
```

```
arg_1: 1
arg_2: 2
arg_3: (3, 4, 5)
```

You can use a for loop to process these other arguments:

```
def example_function(arg_1, arg_2, *arg_3):
    # Let's look at the argument values.
    print('\narg_1:', arg_1)
    print('arg_2:', arg_2)
    for value in arg_3:
        print('arg_3 value:', value)
```

```
example_function(1, 2)
example_function(1, 2, 3)
example_function(1, 2, 3, 4)
example_function(1, 2, 3, 4, 5)
```

Output:

```
arg_1: 1
arg_2: 2
```

```
arg_1: 1
arg_2: 2
arg_3 value: 3
```

```
arg_1: 1
arg_2: 2
arg_3 value: 3
arg_3 value: 4
```

```
arg_1: 1
arg_2: 2
arg_3 value: 3
arg_3 value: 4
arg_3 value: 5
```

We can now rewrite the `adder()` function to accept two or more arguments, and print the sum of those numbers:

```
def adder(num_1, num_2, *nums):
    # This function adds the given numbers together,
    # and prints the sum.
    # Start by adding the first two numbers, which
    # will always be present.
    sum = num_1 + num_2
    # Then add any other numbers that were sent.
    for num in nums:
        sum = sum + num
        # Print the results.
        print("The sum of your numbers is %d." % sum)

# Let's add some numbers.
adder(1, 2, 3)
```

Output:

The sum of your numbers is 6.

In this new version, Python does the following:

- stores the first value in the calling statement in the argument `num_1`;
- stores the second value in the calling statement in the argument `num_2`;
- stores all other values in the calling statement as a tuple in the argument `nums`.

We can then "unpack" these values, using a for loop. We can demonstrate how flexible this function is by calling it a number of times, with a different number of arguments each time.

```
def adder(num_1, num_2, *nums):
    # This function adds the given numbers together,
    # and prints the sum.
    # Start by adding the first two numbers, which
    # will always be present.
    sum = num_1 + num_2
    # Then add any other numbers that were sent.
```

```
for num in nums:
    sum = sum + num
    # Print the results.
    print("The sum of your numbers is %d." % sum)
```

Let's add some numbers.

```
adder(1, 2)
adder(1, 2, 3)
adder(1, 2, 3, 4)
adder(1, 2, 3, 4, 5)
```

Output:

The sum of your numbers is 3.
The sum of your numbers is 6.
The sum of your numbers is 10.
The sum of your numbers is 15.

Accepting an arbitrary number of keyword arguments

Python also provides a syntax for accepting an arbitrary number of keyword arguments. The syntax looks like this:

```
def example_function(arg_1, arg_2, **kwargs):
    # Let's look at the argument values.
    print('\narg_1:', arg_1)
    print('arg_2:', arg_2)
    print('arg_3:', kwargs)
```

```
example_function('a', 'b')
example_function('a', 'b', value_3='c')
example_function('a', 'b', value_3='c', value_4='d')
example_function('a', 'b', value_3='c', value_4='d', value_5='e')
```

Output:

```
arg_1: a
arg_2: b
arg_3: {}
```

```
arg_1: a
arg_2: b
arg_3: {'value_3': 'c'}
```

```
arg_1: a
```

```
arg_2: b
arg_3: {'value_4': 'd', 'value_3': 'c'}
```

```
arg_1: a
arg_2: b
arg_3: {'value_5': 'e', 'value_4': 'd', 'value_3': 'c'}
```

The third argument has two asterisks in front of it, which tells Python to collect all remaining key-value arguments in the calling statement. This argument is commonly named *kwargs*. We see in the output that these key-values are stored in a dictionary. We can loop through this dictionary to work with all of the values that are passed into the function:

```
def example_function(arg_1, arg_2, **kwargs):
    # Let's look at the argument values.
    print('\narg_1:', arg_1)    print('arg_2:', arg_2)
    for key, value in kwargs.items():
        print('arg_3 value:', value)

example_function('a', 'b')
example_function('a', 'b', value_3='c')
example_function('a', 'b', value_3='c', value_4='d')
example_function('a', 'b', value_3='c', value_4='d', value_5='e')
```

Outout:

```
arg_1: a
arg_2: b
```

```
arg_1: a
arg_2: b
arg_3 value: c
```

```
arg_1: a
arg_2: b
arg_3 value: d
arg_3 value: c
```

```
arg_1: a
arg_2: b
arg_3 value: e
arg_3 value: d
arg_3 value: c
```

Earlier we created a function that let us describe a person, and we had three things we could describe about a person. We could include their age, their favorite language, and the date they passed away. But that was the only information we could include, because it was the only information that the function was prepared to handle:

```
def describe_person(first_name, last_name, age=None, favorite_language=None, died=None):
    # This function takes in a person's first and last name,
    # their age, and their favorite language.
    # It then prints this information out in a simple format.
    # Required information:
    print("First name: %s" % first_name.title())
    print("Last name: %s" % last_name.title())

    # Optional information:
    if age:
        print("Age: %d" % age)
    if favorite_language:
        print("Favorite language: %s" % favorite_language)
    if died:
        print("Died: %d" % died)

    # Blank line at end.
    print("\n")

describe_person('brian', 'kernighan', favorite_language='C')
describe_person('ken', 'thompson', age=70)
describe_person('adele', 'goldberg', age=68, favorite_language='Smalltalk')
describe_person('dennis', 'ritchie', favorite_language='C', died=2011)
describe_person('guido', 'van rossum', favorite_language='Python')
```

Output:

First name: Brian
Last name: Kernighan
Favorite language: C

First name: Ken
Last name: Thompson
Age: 70

First name: Adele
Last name: Goldberg
Age: 68
Favorite language: Smalltalk

First name: Dennis
Last name: Ritchie
Favorite language: C
Died: 2011

First name: Guido
Last name: Van Rossum
Favorite language: Python

We can make this function much more flexible by accepting any number of keyword arguments. Here is what the function looks like, using the syntax for accepting as many keyword arguments as the caller wants to provide:

```
def describe_person(first_name, last_name, **kwargs):  
    # This function takes in a person's first and last name,  
    # and then an arbitrary number of keyword arguments.  
    # Required information:  
    print("First name: %s" % first_name.title())  
    print("Last name: %s" % last_name.title())  
    # Optional information:  
    for key in kwargs:  
        print("%s: %s" % (key.title(), kwargs[key]))  
  
    # Blank line at end.  
    print("\n")  
  
describe_person('brian', 'kernighan', favorite_language='C')  
describe_person('ken', 'thompson', age=70)  
describe_person('adele', 'goldberg', age=68, favorite_language='Smalltalk')  
describe_person('dennis', 'ritchie', favorite_language='C', died=2011)  
describe_person('guido', 'van rossum', favorite_language='Python')
```

Output:

First name: Brian
Last name: Kernighan
Favorite_Language: C

First name: Ken
Last name: Thompson
Age: 70

First name: Adele
Last name: Goldberg
Age: 68

Favorite_Language: Smalltalk
First name: Dennis
Last name: Ritchie
Favorite_Language: C
Died: 2011

First name: Guido
Last name: Van Rossum
Favorite_Language: Python

This is pretty neat. We get the same output, and we don't have to include a bunch of if tests to see what kind of information was passed into the function. We always require a first name and a last name, but beyond that the caller is free to provide any keyword-value pair to describe a person. Let's show that any kind of information can be provided to this function. We also clean up the output by replacing any underscores in the keys with a space.

```
def describe_person(first_name, last_name, **kwargs):  
    # This function takes in a person's first and last name,  
    # and then an arbitrary number of keyword arguments.  
    # Required information:  
    print("First name: %s" % first_name.title())  
    print("Last name: %s" % last_name.title())  
    # Optional information:  
    for key in kwargs:  
        print("%s: %s" % (key.title().replace('_', ' '), kwargs[key]))  
  
    # Blank line at end.  
    print("\n")  
  
describe_person('brian', 'kernighan', favorite_language='C', famous_book='The C Programming Language')  
describe_person('ken', 'thompson', age=70, alma_mater='UC Berkeley')  
describe_person('adele', 'goldberg', age=68, favorite_language='Smalltalk')  
describe_person('dennis', 'ritchie', favorite_language='C', died=2011, famous_book='The C Programming Language')  
  
describe_person('guido', 'van rossum', favorite_language='Python', company='Dropbox')
```

Output:

First name: Brian
Last name: Kernighan Famous
Book: The C Programming Language
Favorite Language: C

First name: Ken
Last name: Thompson
Alma Mater: UC Berkeley
Age: 70

First name: Adele
Last name: Goldberg
Age: 68
Favorite Language: Smalltalk

First name: Dennis
Last name: Ritchie

Famous Book: The C Programming Language
Favorite Language: C
Died: 2011

First name: Guido
Last name: Van Rossum
Company: Dropbox
Favorite Language: Python

There is plenty more to learn about using functions, but with all of this flexibility in terms of how to accept arguments for your functions you should be able to write simple, clean functions that do exactly what you need them to do.

Challenges:

- **Absurd Statistics**

Sometimes you just want a simulation that you have absolutely no control over and this program should do exactly that. Make a program that simulates flipping a coin a random number of times. The program should then tell you how many times it landed heads, how many times it landed tails, the total amount of time flipped, and what percentage of the coins were heads or tails. Oh and you should also be able to run it from your console by simply typing “simulation()”

Example Output:

```
simulation()
```

```
Heads: 6
```

```
Tails: 6
```

```
Total: 12
```

```
Heads: 50.0%
```

```
Tails: 50.0%
```

Your Program Should:

- Use multiple functions
- Use a random number generator



Motivate to Innovate...



**STEM
BUILDERS**
Learning Center

MATH | ROBOTICS | TECHNOLOGY | EVENTS | CAMPS



contactus@stembuilders.com



www.stembuilders.com